



AUDITORIA DO SITE E PLANO DE EVOLUÇÃO

Painel administrativo do Movimento 7

Análise pública, revisão de código, prioridades e 15 prompts prontos para implementação

URL analisada: movimento7.onrender.com | Data: 12 de julho de 2026

Documento técnico-operacional • Versão 1.0

1. Resumo executivo

Conclusões mais importantes para decisão e priorização.

O site apresenta uma identidade visual forte, mensagem comunitária clara, formulário funcional, proteção CSRF, banco de dados e uma integração útil com Google Drive. A base é adequada para uma primeira edição do projeto, mas o painel ainda funciona como uma coleção de telas operacionais, e não como um sistema completo de gestão.

Conclusão principal

A maior oportunidade não é refazer o site público. É transformar o painel em uma central de operação com busca, histórico, comunicação, gestão de conteúdo, segurança, auditoria e visão por edição do evento.

Alerta de versão

A página pública acessada em produção apresenta textos diferentes dos existentes no arquivo mais recente revisado. Isso indica possível diferença entre o código implantado e o pacote local. A primeira melhoria deve tornar a versão do deploy visível e rastreável.

Área	Avaliação qualitativa	Leitura
Identidade e narrativa	8/10	Visual forte, proposta compreensível e CTAs claros.
Experiência pública	7/10	Boa base responsiva; faltam data/local, agenda, redes e prova social.
Acessibilidade	7/10	Skip link, alt text e reduced motion; há pontos de contraste e dependência de JavaScript a revisar.
Performance	5/10	Arquivos estáticos somam 8,58 MB; imagens principais estão superdimensionadas.
Administração	4/10	Sem busca, paginação, notas, histórico, edição completa, patrocinadores ou conteúdo.
Segurança	5/10	CSRF e cookies seguros existem; login compartilhado, open redirect e falta de limitação de tentativas são riscos.
Manutenibilidade	5/10	Boa separação Flask, mas sem migrações, testes automatizados e identificação de versão.

As notas são qualitativas e servem para priorização. Não representam um benchmark Lighthouse automatizado, pois a auditoria combinou inspeção da página pública, revisão dos arquivos, testes locais de rotas e validações específicas.

Prioridades imediatas

- Corrigir o redirecionamento aberto no login administrativo.
- Trocar DataRequired por InputRequired no campo de ordem da galeria e definir ordem automática.
- Adicionar edição de título, descrição, texto alternativo, ordem e visibilidade das fotos.
- Adicionar busca, filtros, paginação e tela detalhada das inscrições.
- Implantar migrações de banco e exibir versão/commit do deploy.
- Substituir senha única por contas administrativas com papéis e log de ações.

2. Escopo e método

O que foi verificado e quais limites se aplicam à auditoria.

Foram analisadas as páginas públicas da implantação atual, o comportamento de redirecionamento do painel, o pacote de código mais recente disponível, os registros de implantação e os documentos técnico-visuais do Movimento 7.

Fonte	Verificação realizada
Site público	Home, formulário de inscrição, política de privacidade, login e

Fonte	Verificação realizada
	redirecionamento de /admin/galeria.
Código Flask	app factory, modelos, formulários, rotas, integração Google Drive, templates, CSS, JavaScript e Render.
Testes locais	Compilação Python, respostas HTTP, autenticação, CSV, validação da ordem da galeria e teste de redirecionamento externo.
Identidade	Manual técnico/visual e apresentação institucional fornecidos para o Movimento 7.

Limite de acesso

O painel de produção exige senha. A avaliação interna do painel foi feita pela revisão do código e por execução local autenticada, sem acessar dados reais de inscritos ou credenciais de produção.

Resultado dos testes técnicos locais

Teste	Resultado
Compilação dos módulos Python	Aprovada.
GET /, /inscricao, /privacidade, /saude, /admin/entrar	HTTP 200.
GET /admin sem sessão	HTTP 302 para login.
Login válido e abertura de /admin e /admin/galeria	Funcionando.
Exportação /admin/inscricoes.csv	HTTP 200.
Ordem da galeria = 0	Falha confirmada: "This field is required".
Login com next=https://...	Redirecionamento externo confirmado; deve ser corrigido.

3. Análise da página pública

Conteúdo, navegação, conversão, identidade, desempenho e acessibilidade.

Pontos fortes

- A proposta do Movimento 7 é compreendida rapidamente: cultura de rua, beleza, educação, empreendedorismo, esporte e cuidado coletivo.
- Os botões "Quero participar" e "Preencher inscrição" criam um fluxo de conversão direto para o formulário.
- A seção de parceiros é alimentada pelo banco, possui texto alternativo e suporta links externos.
- O formulário divide as informações em etapas lógicas e oferece política de privacidade vinculada.
- O código já inclui skip link, foco de campos, navegação semântica, lazy loading e respeito a prefers-reduced-motion.
- A identidade utiliza corretamente a paleta escura, roxo, laranja, amarelo e azul definida no manual.

Oportunidades de conteúdo e conversão

Tema	Diagnóstico	Recomendação
Data, local e agenda	A edição é citada, mas a home não destaca data, endereço, horário, programação ou mapa.	Inserir bloco "Próxima edição" administrável.
Galeria real	Na visita pública, a seção exibiu blocos conceituais em vez de fotos.	Mostrar conteúdo real e um estado vazio administrável.
Prova social	Não há números, depoimentos, resultados ou registros de edições anteriores.	Criar métricas e depoimentos aprovados.
Redes sociais	A apresentação institucional cita @movimento7.official, mas o site não destaca a rede.	Adicionar links e feed/CTA social.
Contato	O e-mail contato@movimento7.org deve ser confirmado e gerenciado pelo painel.	Transformar em configuração do site.
Coerência narrativa	A apresentação é centrada em arte/grafite; o site amplia para barbearia, esporte e qualificação.	Unificar a mensagem: arte, beleza e trabalho como ferramentas de transformação.
Navegação	Existe uma seção de galeria, mas o menu principal não possui link direto para ela.	Adicionar "Galeria" ao menu.

Desempenho e experiência técnica

Os arquivos estáticos do pacote somam aproximadamente 8,58 MB. Apenas a logo principal possui cerca de 4,6 MB e o fundo do cabeçalho cerca de 2,17 MB. Para uma landing page, isso aumenta o tempo de abertura em redes móveis e agrava a percepção de cold start do serviço.

- Converter imagens para WebP/AVIF e gerar versões responsivas com srcset.
- Reduzir a logo do cabeçalho para a dimensão realmente exibida e separar favicon da arte principal.
- Definir width e height nos elementos img para reduzir mudança de layout.
- Cachear imagens da galeria fora do fluxo de download do Google Drive a cada requisição.
- Remover style.css.bak do pacote de produção e revisar dependências não utilizadas.

SEO e acessibilidade

- Adicionar canonical, Open Graph, Twitter Cards, sitemap.xml, robots.txt e dados estruturados de Organization/Event.
- Garantir que conteúdo com classe reveal permaneça visível quando o JavaScript não carregar; hoje a visibilidade inicial depende da animação.
- Adicionar aria-current ao menu, mensagens de erro associadas aos campos e títulos de página específicos para cada módulo administrativo.
- Revisar contraste do texto vazado do hero e preservar legibilidade em celulares e modo de alto contraste.
- Adicionar página 404/500 com linguagem do projeto e canal de contato.

4. Revisão do código e da arquitetura

Riscos confirmados, pontos de robustez e recomendações de engenharia.

Pontos positivos do código

- Uso de application factory, extensões desacopladas e Blueprint.
- Proteção CSRF por Flask-WTF e logout via POST.
- Cookies HttpOnly, SameSite=Lax e Secure em produção.
- SQLAlchemy com pool_pre_ping e pool_recycle, útil após erros de conexão observados no Render.
- Otimização das fotos com correção EXIF, redimensionamento e conversão para WebP.
- Refresh token criptografado antes de ser armazenado no banco.
- Exclusão e visibilidade da galeria protegidas por sessão administrativa.

Achados de maior impacto

Prioridade	Achado	Evidência	Ação
P0	Open redirect no login	routes.py:126 usa next sem validar o domínio. O teste redirecionou para URL externa.	Validar destino interno com urlsplit ou ignorar next externo.
P0	Segredos com fallback inseguro	__init__.py usa dev-change-me/admin-change-me quando variáveis faltam.	Falhar ao iniciar em produção se SECRET_KEY ou ADMIN_PASSWORD não existirem.
P0	Ordem 0 inválida	GalleryImageForm usa DataRequired; o valor inteiro 0 é tratado como vazio.	Usar InputRequired e ordem automática.
P1	Sem migrações	db.create_all() roda no startup, mas não altera colunas existentes com segurança.	Adicionar Flask-Migrate/Alembic.
P1	Senha administrativa compartilhada	Não existem usuários, papéis, expiração ou auditoria individual.	Criar AdminUser, hash de senha, RBAC, logout e logs.
P1	Sem paginação	Registration.query.all() carrega todos os registros.	Paginar e filtrar no banco.
P1	Galeria sem edição	O painel oferece apenas ocultar/exibir e excluir.	Criar rota e formulário de edição.
P1	Inconsistência Drive/banco	Upload ou exclusão pode concluir em um sistema e falhar no outro.	Criar compensação, status e fila de reconciliação.
P1	Escopo Google Drive amplo	A integração usa drive completo.	Avaliar drive.file, pasta dedicada ou service account.
P1	CSV vulnerável a fórmulas	Campos iniciados por =, +, - ou	Escapar valores e adicionar

Prioridade	Achado	Evidência	Ação
		@ podem virar fórmula em planilhas.	BOM UTF-8.
P2	Sem versionamento de consentimento	Existe somente booleano lgpd_consent.	Salvar consent_at e privacy_version.
P2	Dependências excessivas	requirements fixa também dependências transitivas e utilitários não usados.	Manter dependências diretas e rotina de atualização.

Riscos específicos da integração com Google Drive

- A chave do token é derivada da SECRET_KEY. Se a SECRET_KEY mudar, o token salvo deixa de ser decifrado. Use uma chave de criptografia separada e versionada.
- gallery_media baixa o arquivo inteiro do Drive para a memória em cada solicitação não atendida pelo cache do navegador.
- Se o upload ao Drive funcionar e o commit do banco falhar, ficará um arquivo órfão.
- Se a exclusão no Drive funcionar e o commit do banco falhar, o registro apontará para um arquivo inexistente.
- O nome do arquivo é montado com o título sem sanitização operacional; convém gerar slug e identificador único.

5. Painel administrativo atual

O que ele resolve hoje e onde gera trabalho manual.

Módulo	O que existe	Limitação operacional
Inscrições	Contagem total/recebidas/aprovadas, filtro por status, detalhes, mudança de status e CSV.	Sem busca, paginação, filtros combinados, notas, atribuição, histórico, ações em lote e comunicação.
Galeria	Upload, conversão WebP, Drive, ordem, ativo, ocultar/exibir e excluir.	Sem edição de metadados, preview, arrastar/soltar, lote, categorias, capa e reconciliação.
Patrocinadores	Exibidos na home por registros do banco.	Não há tela administrativa; alterações dependem de código ou acesso ao banco.
Conteúdo	Textos e contatos ficam em templates.	Qualquer mudança exige editar arquivos e fazer deploy.
Segurança	Senha de ambiente e sessão.	Sem usuários, papéis, 2FA, expiração, tentativas, auditoria e recuperação.
Operação	Health check simples e logs do Render.	Sem versão, alertas, diagnóstico do Drive, backup, status do banco e histórico de falhas.

Estrutura recomendada do novo painel

Menu	Função
Visão geral	Indicadores, pendências, alertas, próxima edição, Drive, banco e backup.
Inscrições	Busca, filtros, paginação, detalhe, notas, histórico, responsáveis e ações em lote.
Galeria	Upload, preview, edição, ordenação visual, categorias, capa e diagnóstico de mídia.
Patrocinadores	Cadastro, logo, link, nível, ordem, ativo e período de exibição.
Conteúdo	Textos da home, contatos, redes, CTA, SEO e blocos publicados.
Edições	Data, local, capacidade, inscrições, programação e status da edição.
Comunicação	Modelos, e-mail/WhatsApp, confirmações e histórico de contato.
Usuários e logs	Papéis, sessões, tentativas, auditoria e ações críticas.
Backup e configurações	Exportações, retenção LGPD, versão, integrações e saúde.

Direção de interface

Manter a linguagem visual do site, mas privilegiar legibilidade e eficiência: sidebar fixa no desktop, navegação recolhível no celular, cards de resumo, tabelas com cabeçalho fixo e visualização em cartões em telas pequenas.

6. Roadmap priorizado

Sequência recomendada para reduzir risco antes de ampliar funcionalidades.

Fase	Prioridade	Entregas	Resultado
Fase 1 — estabilização	P0	Versão do deploy, open redirect, validação de ordem, secrets fail-fast, migrações, edição da galeria, busca/paginação.	Reduz risco e elimina tarefas manuais críticas.
Fase 2 — operação	P1	Novo layout, detalhe de inscrição, notas, histórico, responsáveis, ações em lote, patrocinadores e conteúdo.	Transforma o painel em ferramenta diária.
Fase 3 — governança	P1	Usuários/papéis, auditoria, LGPD, backups, edições do evento e comunicação.	Permite crescimento da equipe com rastreabilidade.
Fase 4 — escala	P2	Cache/CDN de mídia, relatórios, SEO, performance, alertas, testes completos e CI/CD.	Melhora desempenho e manutenção contínua.

Primeiro pacote recomendado para implementação

1. Versão do deploy e página de diagnóstico.
2. Correções de segurança do login e variáveis obrigatórias.
3. Migrações de banco.
4. Edição completa das fotos e correção da ordem.
5. Busca, filtros e paginação das inscrições.
6. Novo shell do painel com sidebar e cards de visão geral.

7. Prompts de implementação

Cada bloco abaixo pode ser copiado e usado como uma tarefa independente de desenvolvimento.

Como utilizar

Execute os prompts na ordem indicada pelas prioridades. Antes de cada alteração, crie backup do banco e uma branch. Ao concluir, rode testes, compile, verifique o layout móvel e entregue o projeto pronto para push.

PROMPT 01 | Versionamento do deploy e diagnóstico

P0

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Eliminar a diferença entre o código local e o que está publicado, exibindo no painel a versão exata em execução e o estado das integrações.

Requisitos obrigatórios

- Adicionar APP_VERSION e GIT_COMMIT por variável de ambiente, com fallback apenas para desenvolvimento local.
- Exibir versão, commit, data do deploy, ambiente, versão Python, banco utilizado e status do Google Drive em /admin/sistema.

- Adicionar a versão em um rodapé discreto de todas as páginas administrativas.
- Criar endpoint /saude detalhado somente para administradores e manter /saude público sem segredos.
- Atualizar render.yaml para preencher o identificador de commit quando a plataforma permitir.

Critérios de aceite

- O administrador consegue confirmar visualmente se o último pacote foi implantado.
- Nenhuma credencial, URL completa de banco ou refresh token é exibida.
- O endpoint público continua respondendo rapidamente com status 200.

Arquivos e camadas provavelmente afetados

app/__init__.py, app/routes.py, app/templates/admin_system.html, app/templates/base_admin.html, render.yaml, README.md

PROMPT 02 | Autenticação segura, papéis e sessões

P0

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Substituir a senha administrativa compartilhada por contas individuais e corrigir vulnerabilidades de sessão e redirecionamento.

Requisitos obrigatórios

- Criar modelo AdminUser com e-mail único, nome, password_hash, role, active, last_login_at e timestamps.
- Usar generate_password_hash/check_password_hash e nunca guardar senha em texto puro.
- Criar papéis administrador, editor e atendimento, aplicando autorização por rota.
- Validar o parâmetro next e aceitar somente caminhos internos; rejeitar URLs externas.
- Adicionar limite de tentativas por IP/usuário, atraso progressivo e log de falhas.
- Definir expiração de sessão, renovação após login, logout de todas as sessões e cookies seguros.
- Criar fluxo seguro para o primeiro administrador via comando CLI, não por senha padrão.

Critérios de aceite

- O teste com next=https://example.com não redireciona para domínio externo.
- Usuários sem papel adequado recebem 403 e não veem botões proibidos.
- Cinco tentativas inválidas em curto período são limitadas e registradas.
- Não existe fallback admin-change-me em produção.

Arquivos e camadas provavelmente afetados

app/models.py, app/forms.py, app/routes.py ou blueprints/auth.py, app/templates/admin_login.html, app/__init__.py, migrations/, tests/

PROMPT 03 | Migrações e configuração segura

P0

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Tornar alterações de banco previsíveis e impedir que a aplicação suba em produção com configuração insegura.

Requisitos obrigatórios

- Adicionar Flask-Migrate/Alembic e gerar migração inicial a partir dos modelos atuais.
- Remover db.create_all() do fluxo de produção; permitir apenas em testes ou comando de desenvolvimento explícito.
- Validar SECRET_KEY, DATABASE_URL e configuração administrativa no startup de produção.
- Criar comandos flask db upgrade e flask seed para implantação e dados iniciais.
- Mover o seed de patrocinadores para comando idempotente, fora da inicialização de cada worker.

- Documentar backup, migração e rollback no README.

Critérios de aceite

- Uma coluna nova é aplicada ao PostgreSQL por migração sem apagar dados.
- A aplicação recusa iniciar em produção quando faltam segredos obrigatórios.
- O seed pode ser executado repetidamente sem duplicar registros.

Arquivos e camadas provavelmente afetados

requirements.txt, app/__init__.py, app/models.py, migrations/, render.yaml, README.md

PROMPT 04 | Novo layout e visão geral do painel

P1

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Criar uma navegação administrativa consistente, responsiva e orientada a tarefas.

Requisitos obrigatórios

- Criar base_admin.html separada do layout público, com sidebar e menu móvel.
- Adicionar rotas Visão geral, Inscrições, Galeria, Patrocinadores, Conteúdo, Edições, Usuários, Logs e Configurações conforme permissões.
- Na visão geral, exibir totais por status, novas inscrições nos últimos 7 dias, pendências, fotos visíveis, Drive, banco, último backup e versão.
- Adicionar breadcrumbs, título da página, ações principais e estado ativo do menu.
- No celular, transformar tabelas extensas em cartões ou permitir colunas prioritárias sem rolagem confusa.
- Manter contraste, foco visível, teclado e identidade do Movimento 7.

Critérios de aceite

- Todas as páginas administrativas compartilham cabeçalho, menu, mensagens e rodapé.
- O painel funciona em 360 px de largura sem sobreposição.
- A página inicial administrativa mostra pendências operacionais em menos de uma tela.

Arquivos e camadas provavelmente afetados

app/templates/base_admin.html, app/templates/admin_dashboard.html, app/static/css/admin.css, app/static/js/admin.js, app/routes.py

PROMPT 05 | Busca, filtros e paginação de inscrições

P0

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Permitir encontrar rapidamente qualquer inscrição sem carregar toda a tabela na memória.

Requisitos obrigatórios

- Adicionar busca por nome, nome social, e-mail, telefone e bairro.
- Adicionar filtros por status, tipo de participação, cidade, disponibilidade, data inicial/final e edição do evento.
- Implementar paginação no banco com 25 itens por página e opções 25/50/100.
- Permitir ordenação por data, nome e status com parâmetros validados.
- Preservar filtros ao mudar status e ao voltar da tela de detalhes.
- Mostrar rótulos humanos para participation_type e availability, não os slugs crus.
- Exibir contador de resultados e botão "limpar filtros".

Critérios de aceite

- Nenhuma chamada da listagem usa .all() sem limite.

- A busca combinada retorna resultados corretos e mantém parâmetros na paginação.
- Filtros inválidos não geram erro 500.

Arquivos e camadas provavelmente afetados

app/routes.py, app/models.py, app/templates/admin_dashboard.html, app/static/css/admin.css, tests/test_admin_registrations.py

PROMPT 06 | Detalhe, notas, responsável e histórico

P1

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Transformar cada inscrição em um processo acompanhável por toda a equipe.

Requisitos obrigatórios

- Criar tela /admin/inscricoes/<id> com todos os dados, links clicáveis e resumo lateral.
- Adicionar campos assigned_to, priority e updated_at.
- Criar modelos RegistrationNote e RegistrationStatusHistory com autor e data.
- Toda mudança de status deve registrar status anterior, novo status, usuário e observação opcional.
- Permitir notas internas, fixar nota importante e editar apenas pelo autor ou administrador.
- Adicionar botões de e-mail, WhatsApp e copiar contato, com sanitização do telefone.
- Converter datas UTC para America/Sao_Paulo na exibição.

Critérios de aceite

- O histórico é imutável e mostra quem realizou cada alteração.
- Notas internas nunca aparecem no site público nem no CSV padrão.
- O responsável e a prioridade podem ser filtrados na listagem.

Arquivos e camadas provavelmente afetados

app/models.py, app/routes.py, app/forms.py, app/templates/admin_registration_detail.html, migrations/, tests/

PROMPT 07 | Ações em lote e central de comunicação

P1

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Reduzir tarefas repetitivas na seleção e no contato com participantes.

Requisitos obrigatórios

- Adicionar seleção múltipla na listagem e ações em lote para status, responsável, tag e exportação.
- Exigir confirmação com quantidade de itens antes de alterações destrutivas ou de status.
- Criar modelos de mensagem por situação: recebida, em análise, aprovada, lista de espera e recusada.
- Gerar links de WhatsApp e e-mails pré-preenchidos; integração de envio automático deve ser opcional e configurável.
- Registrar CommunicationLog com canal, destinatário, modelo, autor, data e resultado.
- Evitar exposição de todos os e-mails em CC; usar envios individuais ou BCC quando apropriado.

Critérios de aceite

- Uma seleção de 20 inscrições pode mudar de status com um único fluxo confirmado.
- A comunicação fica registrada por participante.
- Falhas parciais não duplicam mensagens em nova tentativa.

Arquivos e camadas provavelmente afetados

app/models.py, app/routes.py, app/templates/admin_dashboard.html,

PROMPT 08 | Edição completa das fotos da galeria

P0

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Permitir editar título, descrição, texto alternativo, ordem e visibilidade sem reenviar a imagem.

Requisitos obrigatórios

- Criar GalleryImageEditForm sem FileRequired e rota GET/POST /admin/galeria/<id>/editar.
- Adicionar botão “Editar” em cada card da galeria.
- Corrigir display_order usando InputRequired em vez de DataRequired, aceitando zero.
- Ao abrir a edição, preencher todos os valores atuais e validar limites.
- Atualizar updated_at e registrar a alteração em AuditLog.
- Após salvar, voltar para a posição da foto na lista e mostrar feedback claro.
- Não alterar nem reenviar o arquivo no Google Drive quando apenas metadados forem modificados.

Critérios de aceite

- Título, descrição, alt text e ordem são alterados e aparecem imediatamente na home.
- Ordem 0 é aceita.
- Usuário sem permissão de editor recebe 403.

Arquivos e camadas provavelmente afetados

app/forms.py, app/routes.py, app/models.py, app/templates/admin_gallery.html, app/templates/admin_gallery_edit.html, migrations/, tests/test_gallery.py

PROMPT 09 | Ordenação visual, upload em lote e robustez de mídia

P1

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Tornar a galeria rápida de organizar e resistente a falhas entre Google Drive e PostgreSQL.

Requisitos obrigatórios

- Adicionar drag-and-drop com botão “Salvar ordem” e atualização transacional de posições sequenciais.
- Permitir upload de múltiplas fotos com preview, título sugerido, alt text obrigatório e progresso por arquivo.
- Gerar slug/UUID para nomes no Drive e armazenar tamanho, largura, altura, hash e updated_at.
- Quando o upload ao Drive funcionar e o banco falhar, excluir o arquivo órfão automaticamente.
- Quando a exclusão do Drive falhar, marcar estado pending_delete e oferecer nova tentativa.
- Criar rotina de reconciliação que detecte registros sem arquivo e arquivos sem registro.
- Adicionar thumbnail e cache local/externo para não baixar a imagem completa do Drive a cada acesso.

Critérios de aceite

- A ordem publicada corresponde exatamente à ordem exibida no painel.
- Falha simulada após upload não deixa arquivo órfão.
- O painel mostra progresso e erro individual em lote, sem perder uploads concluídos.

Arquivos e camadas provavelmente afetados

app/models.py, app/routes.py, app/google_drive.py, app/static/js/admin_gallery.js, app/templates/admin_gallery.html, migrations/, tests/

PROMPT 10 | Gestão de patrocinadores

P1

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Permitir que a equipe administre parceiros sem editar código ou acessar o banco.

Requisitos obrigatórios

- Criar CRUD de patrocinadores com nome, descrição, site, logo, ordem, ativo, categoria/nível e período de exibição.
- Validar URL e imagem, otimizar a logo preservando transparência e gerar preview.
- Adicionar ordenação visual e alternância de visibilidade.
- Não excluir imediatamente patrocinador já usado em edição passada; preferir inativação ou soft delete.
- Permitir vincular patrocinador a uma ou mais edições.
- Remover o seed automático do startup e manter comando de importação inicial.

Critérios de aceite

- Um patrocinador pode ser criado, editado, ordenado e ocultado pelo painel.
- A home exibe somente registros ativos e válidos para a edição publicada.
- Links externos usam noopener/noreferrer e logos têm alt text.

Arquivos e camadas provavelmente afetados

app/models.py, app/forms.py, app/routes.py, app/templates/admin_sponsors*.html, app/services/media.py, migrations/, tests/

PROMPT 11 | CMS de conteúdo e edições do evento

P1

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Permitir atualizar textos, contatos, redes, data, local, agenda e estado das inscrições sem novo deploy.

Requisitos obrigatórios

- Criar modelo EventEdition com nome, slug, número, data/hora, local, endereço, capacidade, status e período de inscrições.
- Vincular Registration, GalleryImage e Sponsor a uma edição.
- Criar SiteContent/SiteSetting para textos do hero, CTAs, contatos, redes sociais, SEO e blocos da home.
- Adicionar rascunho, pré-visualização e publicação, mantendo versão anterior para rollback.
- Usar campos estruturados em vez de HTML livre sempre que possível; sanitizar qualquer rich text permitido.
- Exibir automaticamente a próxima edição publicada e fechar formulário fora do período configurado.

Critérios de aceite

- A equipe altera data, local, e-mail e Instagram sem tocar em templates.
- Inscrições de edições diferentes não se misturam.
- Conteúdo em rascunho não aparece publicamente e pode ser pré-visualizado por administrador.

Arquivos e camadas provavelmente afetados

app/models.py, app/routes.py, app/forms.py, app/templates/admin_content*.html, app/templates/admin_editions*.html, app/templates/index.html, app/templates/register.html, migrations/, tests/

PROMPT 12 | LGPD, retenção, exportação e backup

P1

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Alinhar o que a política promete com recursos reais de correção, exclusão e retenção.

Requisitos obrigatórios

- Adicionar `consent_at`, `privacy_version` e `edition_id` à inscrição.
- Criar ação de corrigir dados, anonimizar e excluir, com confirmação forte e log de auditoria.
- Definir política de retenção por edição e listar registros próximos do prazo.
- Criar exportação CSV e XLSX com filtros atuais, BOM UTF-8 e proteção contra CSV injection em valores iniciados por `=`, `+`, `-` ou `@`.
- Permitir backup administrativo com data, quantidade, checksum e armazenamento seguro.
- Separar exportações operacionais de exportações completas com dados sensíveis e exigir permissão específica.

Critérios de aceite

- É possível atender um pedido de correção ou anonimização a partir do painel.
- Planilhas abrem com acentos corretos e não executam fórmulas vindas de campos de usuário.
- A política de privacidade descreve exatamente os fluxos implementados.

Arquivos e camadas provavelmente afetados

`app/models.py`, `app/routes.py`, `app/services/exports.py`, `app/templates/admin_privacy.html`, `app/templates/privacy.html`, `migrations/`, `tests/`

PROMPT 13 | Auditoria, saúde e observabilidade

P1

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Dar visibilidade às ações administrativas e detectar falhas antes que afetem o evento.

Requisitos obrigatórios

- Criar AuditLog com usuário, ação, recurso, `resource_id`, resumo seguro, IP, user agent e timestamp.
- Registrar login, falha de login, status, notas, conteúdo, galeria, patrocinadores, exportações, exclusões e configurações.
- Criar painel de logs com filtros e acesso exclusivo de administrador.
- Separar liveness e readiness: processo, banco e, opcionalmente, Google Drive.
- Adicionar logs estruturados com `request_id` e integração opcional com serviço de erros.
- Exibir alertas no dashboard para Drive desconectado, backup vencido, migração pendente e falhas recentes.

Critérios de aceite

- Toda ação crítica identifica autor e horário.
- Tokens, senhas e dados completos de inscrição não aparecem nos logs.
- Falha do banco faz readiness falhar sem derrubar o endpoint de liveness.

Arquivos e camadas provavelmente afetados

`app/models.py`, `app/middleware.py`, `app/routes.py`, `app/templates/admin_audit.html`, `app/services/health.py`, `render.yaml`, `migrations/`, `tests/`

PROMPT 14 | Performance, SEO e acessibilidade

P2

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Reduzir peso da página e melhorar descoberta, estabilidade visual e uso por tecnologias assistivas.

Requisitos obrigatórios

- Converter logo, hero e patrocinadores para WebP/AVIF, mantendo originais fora do pacote servido.
- Gerar srcset/sizes, dimensões explícitas, thumbnails e política de cache com fingerprint de arquivos.
- Adicionar canonical, Open Graph, Twitter Cards, sitemap, robots e JSON-LD de Organization/Event.
- Adicionar link Galeria ao menu e aria-current nas páginas/âncoras.
- Alterar animação reveal para conteúdo visível por padrão e animar somente após JavaScript adicionar classe de suporte.
- Revisar contraste, foco, labels, mensagens de erro, teclado e reduced motion.
- Criar páginas 404/500 e estado vazio da galeria com CTA administrativo.

Critérios de aceite

- O pacote estático inicial é significativamente menor que 8,58 MB.
- A página continua legível com JavaScript bloqueado.
- Nenhuma imagem sem dimensões ou texto alternativo é renderizada no conteúdo principal.

Arquivos e camadas provavelmente afetados

app/templates/base.html, app/templates/index.html, app/static/css/style.css, app/static/js/site.js, app/static/img/, app/routes.py, tests/all.py

PROMPT 15 | Testes automatizados e CI/CD

P1

Prompt copiável

Você é responsável por evoluir o projeto Movimento 7, uma aplicação Flask 3 com Jinja2, Flask-SQLAlchemy, PostgreSQL no Render, SQLite local, Flask-WTF/CSRF e integração com Google Drive. Implemente a alteração abaixo sem remover rotas existentes, sem expor segredos e preservando a identidade visual do Movimento 7 (fundo escuro, roxo, laranja, amarelo e azul, tipografia de impacto e linguagem comunitária).

Objetivo

Evitar regressões no formulário, painel, migrações e integração de mídia antes de cada push.

Requisitos obrigatórios

- Adicionar pytest com factory de teste, SQLite temporário e CSRF configurável.
- Cobrir páginas públicas, formulário válido/inválido, autenticação, autorização, open redirect, filtros, status, CSV, galeria, patrocinadores e LGPD.
- Mockar Google Drive e testar upload, falha, compensação e exclusão sem usar credenciais reais.
- Adicionar ruff ou equivalente, formatter, verificação de templates e auditoria de dependências.
- Criar workflow de CI que rode lint, testes, migrações e build em cada pull request/push.
- Adicionar smoke test pós-deploy para /saude, home, login e versão esperada.

Critérios de aceite

- O pipeline falha quando o redirecionamento externo volta a ser aceito ou quando ordem 0 é rejeitada.
- Nenhum teste depende de internet ou de banco de produção.
- O deploy só é considerado concluído quando a versão publicada corresponde ao commit esperado.

Arquivos e camadas provavelmente afetados

tests/, pyproject.toml, requirements-dev.txt, .github/workflows/ci.yml, app/, migrations/, README.md

8. Checklist de aceite do projeto evoluído

Critérios de conclusão para uma versão operacionalmente madura.

- Versão e commit visíveis no painel e coerentes com o deploy.
- Login sem redirecionamento externo, senha padrão ou tentativas ilimitadas.
- Migrações reproduzíveis em banco vazio e banco com dados.
- Inscrições com busca, filtros, paginação, detalhe, notas, responsável e histórico.
- Galeria com edição, ordem visual, preview, lote e reconciliação Drive/banco.
- Patrocinadores, conteúdo e edições gerenciáveis sem alterar código.
- Exportações protegidas contra fórmulas e com acentos corretos.

- Fluxos de LGPD, retenção e anonimização disponíveis e auditados.
- Painel utilizável em desktop e celular, com teclado e foco visível.
- Testes e CI aprovados antes do push/deploy.

Recomendação final

Implemente primeiro os prompts 01, 02, 03, 05 e 08. Eles resolvem a divergência de deploy, os riscos de segurança, a evolução de banco, a escala da listagem e a principal lacuna atual da galeria.

Fontes analisadas

Site público: <https://movimento7.onrender.com>

Código: pacote movimento7_site_corrigido_drive.zip, com referência adicional aos arquivos routes.py e __init__.py fornecidos.

Identidade e documentação: Manual_Movimento7(1).pdf e Apresentação com modelo fornecido_260711_184451.pdf.

Registros: logs de implantação do Render fornecidos durante o desenvolvimento, incluindo erros históricos de conexão PostgreSQL, OAuth e validação da ordem da galeria.



UMA COMUNIDADE EM MOVIMENTO

Segurança, organização e autonomia para a equipe do Movimento 7.